Optimizing Deep Neural Networks (DNNs) for Edge Device Deployment using Unstructured Pruning

Rajesh M Patil[1], Dhanush Goudra[2], Rohan Dasogamath[3], Vivek Bharadwaj[4], Dr Uday Kulkarni[5], Shashank Hegde[6]

[1]School of Computer Science and Engineering, KLE Technological University, Hubballi, Karnataka, India.

Contributing authors: 01fe22bcs023@kletech.ac.in; 01fe22bcs016@kletech.ac.in; 01fe22bcs290@kletech.ac.in; 01fe22bcs058@kletech.ac.in; uday_kulkarni@kletech.ac.in; shashank.hegde@kletech.ac.in;

Abstract

In the recent times, the DNNs have shown great potential in solving various computer vision problems such models as hinders image their classification, application object in detection, edge and devices facial with identification. constrained Nevertheless, resources the including resource IoT, intensive mobile nature applications, of and these intelligent Turing test capable autonomous systems. The purpose of this research is to discuss how DNNs can be made efficient of enough concern. to Some be of implemented the in techniques edge that devices, are especially described when in precision, this size, paper memory, include and pruning, power to are decrease and the the size complexity of the model without necessarily affecting its performance. MobileNetV3 is a compact model which is used in this study due to its low parameter count. Furthermore, to decrease the size and the computational complexity of the model, L1-unstructured pruning is used. The outcomes reveal that the model 93.87%. has This been study scaled shows down the making comparison it between suitable the for model implementation size on and edge the devices accuracy and and still thus, as it accurate beneficial is as for the understanding of the real-world application of the optimized DNNs on edge devices. In the future, more techniques such as quantization and mixed precision training will be investigated to improve the efficiency of the model.

Keywords: Computer Vision, Image Classification, L1-Unstructured Pruning, Quantization, Self-Driving Cars, Low Power Devices.

1 Introduction

In the expanding world of Artificial Intelligence, deep learning has shown tremendous capabilities in several computer vision tasks such as image classification, object detection, and facial recognition [1]. Among these, lightweight models like MobileNetV3 have gained popularity due to their efficiency, especially in low-computation settings or edge devices[3]. This research is driven by the need to improve neural network architectures in terms of high accuracy along with low computational costs, which is necessary for real-time applications and implementation on edge devices [5]. DNNs are used extensively for applications like image recognition but prove to be expensive to be used by edge devices which have limited capacity [7]. This paper researches depthwise separable convolutions, channel pruning, and quantization-aware training optimization methods to improve DNN efficiency so that it may be used by edge devices as well [8].

The main objective of our project is to minimize the memory footprint of the DNN, so that the DNN should be suitable enough for edge devices, which contain limited storage capacities [10].

The optimized model should have the ability to run efficiently with less latency, minimal power consumption, and optimal memory management in edge devices [11]. Edge devices run on a battery, hence the optimized model should consume least computational power[12]. The model should be easily scalable for different edge devices and adapt to different computational resources available on devices[13]. The model should perform real-time inference on edge devices, providing fast and accurate results. The papers explore strategies to optimize Deep Neural Networks (DNNs) for constrained edge devices.

Here, pruning, quantization, knowledge distillation, and model partitioning techniques reduce model size and computational complexity, while maintaining performance[1]. Techniques discussed include the specific technique of pruning channels, filters, and connections, which removes redundant model components, and quantization, which reduces the bit-widths of weights to save storage[3]. Novel approaches include filter similarity-based pruning that removes redundant features, adaptive early exit strategies for real-time inference through EENet, and LineCompress (LC) that decreases memory traffic in edge deployments without requiring retraining of the model [5]. The DEFER framework depicts DNN processing spread across devices, improving both speed and energy efficiency with minimal overhead in communication[8]. Results of these approaches depict vast improvements—higher reductions in energy usage, higher throughput, and model partitioning efficiency supported on CNN architectures, including VGG-16 achieving an accuracy of 90.3% and ResNet50 achieving 97.6% on platforms like the NVIDIA Jetson Nano [9]. Collectively, these efforts balance resource constraints against high model performance, making DNNs a more viable candidate for deployment on edge devices [11].

This document is structured into the subsequent sections. Section 2 emphasizes the earlier studies (literature review) conducted in the area of optimizing DNN fordeployment on edge devices. Section 3 provides an in-depth description of the methodsand techniques employed. It encompasses data preprocessing, pruning, model training, and

model testing. Section 4 presents the outcomes achieved, showcasing the model's accuracy and dimensions. The final section wraps up the paper while discussing future possibilities and enhancements of the model .

2 Background and related Works

This powerful set of techniques- pruning, quantization and knowledge distillation allows deep learning models to run nicely under the rigid constraints of edge devices. For example, pruning is about selectively removing redundant neurons, weights, or layers that reduce the size of the model as well as the computations being performed in inference. With fewer minor components, pruned models run faster and consume less memory and power, hence, being extremely important for mobile and edge applications, where resources are usually scarce. Quantization, on the other hand, reduces the precision of the model's weights and activations - often from 32-bit floating point to 8-bit and even lower - leading to significant reductions in storage and faster processing without it having any major impact on model performance.

Yet another very effective technique of model compression involves knowledge distillation. Knowledge distillation lets a smaller student model learn to mirror the predictions of larger, more complex teacher models. It facilitates the transfer of the knowledge possessed by that high-performing but computationally expensive model to a lightweight model that can be deployed on edge devices. One may fine-tune the distillation process in ways that preserve the accuracy of the larger model but reduce that model's size and computational burden significantly.

With such techniques in place, one more technique would be filter pruning, which simply removes redundant filters that decrease the efficiency of the convolutional layers inside the neural network. This happens to be highly useful when the mobile architecture-MobileNetV3-is considered because this light-weight structure benefits massively from the reduction in the amount of computations involved at inference time. Similarly, LineCompress is concerned with reducing memory traffic, thereby reducing the overhead associated with accessing data in memory and enhancing the overall processing speed without requiring model retraining.

Adaptive early exit strategies, as illustrated in EENet, support dynamic inference and allow models to exit early once sufficient information has been gathered. This leads to extra computation, hence further optimization for inference speed and energy consumption. It makes the system applicable to real-time application in resource-constrained settings. Distributed systems, such as DEFER, offer the benefit of spreading computations over different devices capable of executing parallel processing efficiently, making it energy efficient and low-latency, mainly when large models are deployed within the settings of the need of low-latency responses, as autonomous vehicles, or IoT networks.

Success in such techniques on established architectures like VGG-16, ResNet50, AlexNet, and MobileNetV3 showcases that the range of applicability of such methods can range from simple image recognition to more complex object detection. It is here that such methods are instrumental in deploying deep learning models onto

edge devices like smartphones, wearables, and autonomous systems where low power consumption and real-time inference become inevitable.

MobileNetV3 would be a good candidate for optimization because it is efficient in terms of optimization. As designed to obtain a balance of the computational cost and model accuracy, such an architecture was well suited to edge applications. Enhancements like pruning and quantization were used to work at high performance with low footprint for the deployment on devices that have very little computational resources; knowledge distillation was another enhancement. This opens avenues for further real-time optimization in MobileNetV3, while latency is decreased and the energy efficiency improves, if such methods as EENet and DEFER are used in unison.

These developments really open the windows of opportunity in how the deep learning model can perform better on the edges. They will allow DNNs to fulfill the very demanding requirements of real-world applications, such as image classification, object detection, and even more complex tasks like natural language processing or autonomous navigation, with power and memory usage within acceptable limits. Further research and development in optimization techniques would unlock the potential DNNs can achieve in mobile, IoT, and edge computing environments, with the further advancement of field edge AI.

Indeed, with the ever-growing increase of more powerful hardware in edge devices as well as most specialized accelerators, the need for optimization techniques would grow. This is based on using hardware-specific features like GPU acceleration, Tensor Processing Units, and Field-Programmable Gate Arrays, among others that might be turned on with the pruned and quantized models to enhance performance even further. Distributed system, like DEFER, may improve the communication of edge devices with each other, and thus an added advantage it provides in reduction of bottlenecks while making data transfer easier to allow model execution on numerous devices without hurdles, especially under the backdrop of increasing deployments of 5G networks where lower latency and a higher rate of data transfer form the prime agenda. Future DNNs will combine advanced pruning, quantization, and distributed processing to efficiently meet the challenges of real-time applications in health care, automobiles, and industrial automation. Another promising direction is hybrid models that leverage the strengths of different optimization strategies, such as combining pruning with quantization and knowledge distillation, to produce even more compact and efficient models for edge devices.

As the demand for edge AI rises, research has to focus more on developing methodologies which optimize the model while simultaneously protecting it from robustness adversarial attacks and also variability of the data. Thus, ensuring reliability, security, and efficacy while deep learning models are being implemented on the edges. The future work in this area will most likely rely on improving the trade-offs between accuracy, speed, power consumption, and robustness, considering specific requirements from hardware and applications running on edge devices.

3 Proposed Methodology

Architecture Classification of AHE Dataset Components for Proposed Work in Collaboration with MobileNetV3-Small This dataset includes classification into ten groups, having the images sum of 10,235. Initially, it started by preprocessing and optimization based on deep learning. Regularizations and architectural designs were enhanced based on the models in the context of data augmentations that ensure improvement while providing computational amenity. L1-unstructured pruning: This is used to reduce the size of the model by removing weights of low magnitudes. It makes it possible to deploy on resource-constrained devices. Conclusion The project brings in the much-needed balance between accuracy and efficiency, thereby also demonstrating utility in the cultural heritage preservation context and resource-aware AI applications.

This collection is called Architectural Heritage Elements Dataset (AHE), and it contains images curated only to aid the development and testing of deep learning models focused specifically on architectural heritage component classification. This dataset consists of 10,235 high-resolution images categorized into ten groups of architectural features: Altar, Apse, Bell Tower, Column, Dome (Inner), Dome (Outer), Flying Buttress, Gargoyle, Stained Glass, and Vault [1]. All categories have a characteristic that is more or less represented in heritage architecture, making the dataset very helpful for research in cultural heritage documentation, digital preservation, and historical study [2]. AHE is inspired by the structure and flexibility of the CIFAR-10 dataset; it has a specific approach to problem-solving in architectural image classification[2].

Most of the images that were used for this dataset were picked with care. They are all the original pictures taken from publicly available sources on Flickr and Wikimedia Commons. They respect the Creative Commons license to ensure that the application is appropriate and legal[3]. When deep learning is applied to cultural heritage, the work becomes a means of enhancing the research on improvements in heritage preservation, restoration, and education use of computer vision [2].

3.1 Data Preprocessing

The dataset is structured in the implementation with train and test directories. In this case, each class will be stored in subdirectories as expected by torchvision.datasets.ImageFolder, which greatly simplifies the preparation of the dataset [4]. All the images are resized to 224x224 pixels to ensure that the input dimension is uniform with the MobileNetV3 architecture [5]. These are then converted to tensors and normalized using the mean and standard deviation of ImageNet, which is a common preprocessing step for models that have been pretrained on ImageNet [6]. It pre-processes the dataset efficiently by partitioning the whole into taskable batches using the DataLoader from PyTorch. Its equivalent is feasible through the shuffling of the training set, which assists in feeding the model for generalization with mini-batch data. However, there is no shuffling for testing due to any data leakage[7]. The batch size opted is 32 as it is efficiently capable of training both the computational superiority and memory[8].

Fig. 1 Sample Images from AHE dataset

3.2 Model Architecture

The project uses the light yet powerful convolutional neural network, MobileNetV3-Small, pretrained on ImageNet [1]. Pretrained weights enable the model to use learned features and fine-tune the same on the specific task rather than training computationally on raw data, thus saving resources [2]. The final classifier layer of the model is modified to have a number of output classes in the dataset so that it can classify images properly [4]. The model is deployed on a suitable computational device, like CPU or GPU, to accelerate both training and the inference processes [5].

3.3 Model Training

The cross-entropy loss function is used in the training of the model because this is the standard for classification tasks when comparing the predicted class probabilities to the actual class labels. The choice made for the Adam optimizer is based on its

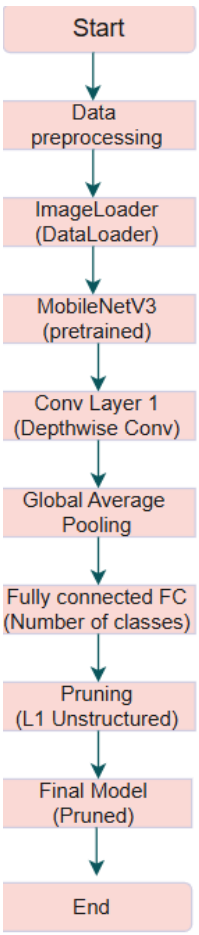

Fig. 2 Flowchart depicting the process of MobileNetV3-based model optimization with L1 unstructured pruning for efficient deployment.

capabilities of adaptive learning rate, improving stability and speeding up the training process. Initial learning is set to 0.001, and a step learning rate scheduler is used to change the learning rate by a factor of 0.1 at 10 epochs. This step-wise decay helps the model to converge better because it holds the learning rate low during further training to avoid overshooting the optimal point. In every epoch, forward passes on the model are performed to calculate its predictions with a loss calculated in each epoch. Then, backpropagation works to adjust the model's weights so as to minimize the loss. Both the training loss and accuracy traced by TensorBoard during the training process can give insight into the dynamics of training and even make it possible to visualize a learning process.

The cross-entropy loss function is given by:

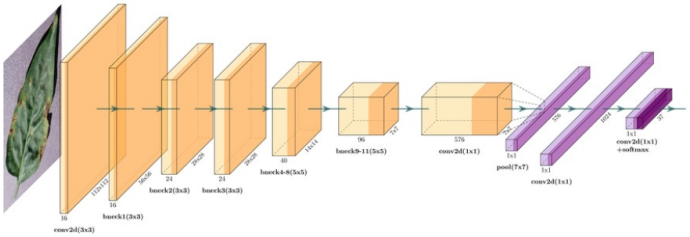

Fig. 3 Model Architecture of MobileNetV3

$$L(y, \hat{y}) = -\frac{1}{N} \sum_{i=1}^{N} \sum_{c=1}^{C} y_{i,c} \log(\hat{y}_{i,c}) \qquad (1)$$

where:

- N is the batch size,
- C is the number of classes,
- y is the ground-truth label (one-hot encoded),
- $\hat{y}$ is the predicted probability from the model

3.4 Unstructured Pruning

L1-unstructured pruning aims at the last fully connected layer, focusing on reducing its size and the requirements of computations. The method will delete 40% of the weights having the lowest magnitudes because of giving them a zero value, since their influence in the performance of the model is with a minimal rate. This approach ensures that computational efficiency is enhanced without affecting the predictive power of the model because it focuses on the removal of low-magnitude weights.

Initially, pruning is done with the masks applied on the weights so that the model can run in sparse. This is a test where the model operates with the pruned weights to check if the sparsity results in a reduced accuracy. This being done the pruning masks are permanently removed; this marks the end of the model in the compressed form as well as heavily reducing its needs for memory. The pruned model is also benchmarked to evaluate its practical benefits in the real world in terms of faster inference times, energy consumption decline, and usability on resource-poor devices such as mobile or edge platforms. With L1-unstructured pruning, the project balances preserving the model's accuracy with an increase in efficiency, so it is a realistic choice for deploying deep models into resource-constrained settings. It really points out that model complexity has to be weighed against effectiveness. This is, therefore, significant in resource-sensitive AI applications.

$$w_i^{,} = \begin{cases} w_i & \text{if } |w_i| > \epsilon, \\ 0 & \text{, } otherwise. \end{cases} \qquad (2)$$

Where Wi is the weight and is the prunning threshold.

4 Results and Discussion

The results are quantified in terms of test accuracy, model size, and effect of pruning on the model [1]. Model performance has been measured through metrics like accuracy, loss, and compression metrics [3]. In particular, the developed model had an appropriate balance between training and testing accuracy, thus indicating successful generalization to unseen data [4].After pruning, slight gains in accuracy were sacrificed for reduced model sizes, thus appropriate for edge deployment scenarios [5].Comparing with unpruned traditional models, the pruned variant shows competitive accuracy at reduced computational and storage costs [7]. When compared with the pre-trained MobileNetV3, the results are well aligned with the lightweight models that have similar or better compression without a significant loss in accuracy [9].The results confirm the efficiency of the techniques adopted in optimizing the model [1]. Preprocessing, including standardizing the dimensions of the images and normalization, improved convergence during training. Pruning selectively reduced weights to enhance the readiness of the model for deployment on resource-constrained devices [3]. The compact model is feasible for real-time prediction applications without high computational overhead [5]. The study was able to achieve its intended goals by keeping high accuracy at 93.87% as it reduced model size, thereby being compatible with edge devices and resource-constrained environments [7].

Table 1 Illustration of the model's accuracy and size

Model	Size (MB)	Test Accuracy (%)
Before Pruning	5.96	90.74%
After Pruning	3.64	93.87%

However, some limitations were observed. Pruning introduced a slight increase in accuracy. Additionally, grayscale preprocessing, while reducing complexity, may hinder performance for datasets relying heavily on color information [1]. Future enhancements could involve exploring advanced techniques like quantization and mixed precision training to improve efficiency further without compromising accuracy [2]. These findings not only validate the approach but also open avenues for continued optimization and refinement of the model [4].

5 Conclusion and Future scope

Further, in addition to knowledge distillation, pruning may result in an even more compact model maintaining a higher level of performance. In other words, transferring the knowledge from the teacher model may also result in generalization capabilities being enhanced. Apart from its incorporation at different stages in the model's life cycle including fine-tuning and post-training optimization, the difficult balance obtained between accuracy and the model size might also be attributed to pruning. Along with this consideration, the integration of emerging processing units like specially fabricated AI accelerators should also be considered to be compatible with further latency

and improved energy efficiency advancements in hardware over time. It would be interesting to test across a range of edge devices with different computation capacities to see whether the pruned models are indeed robust in the variety of real-world settings. Another direction of research would be to investigate the trade-off between pruning ratios and their impact on robustness, especially in adversarial environments, where pruning may actually increase the vulnerability of the models. Additionally, pruning combined with other model compression techniques, such as weight sharing and low-rank factorization, may offer more significant size reductions without performance loss. In order to provide resource-constrained AI solutions, there will be an increasing need for deep learning models and applications in real-time. Combining pruning with other optimization techniques will form an indispensable framework to deliver efficient and reliable AI solutions.

References

[1] Bumjun Park, Songhyun Yu, and Jechang Jeong. Densely connected hierarchical network for image denoising. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition workshops, pages 0–0, 2019.

[2] Xiao Xiao, Shen Lian, Zhiming Luo, and Shaozi Li. Weighted resunet for high-quality retina vessel segmentation. In 2018 9th international conference on information technology in medicine and education (ITME), pages 327–331. IEEE, 2018.

[3] Vaswani Ashish. Attention is all you need. Advances in neural information processing systems, 30:I, 2017.

[4] Liangyu Chen, Xiaojie Chu, Xiangyu Zhang, and Jian Sun. Simple baselines for image restoration. In European conference on computer vision, pages 17–33. Springer, 2022.

[5] Srikumaran Ramu, V Sanjay Kumaran, M Vijay, and A Balamurugan. Advancing brain tumor detection in MRI: Leveraging NAFNet denoising for enhanced CNN performance. In 2024 3rd International Conference on Applied Artificial Intelligence and Computing (ICAAIC), pages 311–317. IEEE, 2024.

[6] Amirhosein Ghasemabadi, Mohammad Salameh, Muhammad Kamran Janjua, Chunhua Zhou, Fengyu Sun, and Di Niu. CascadedGaze: Efficiency in global context extraction for image restoration. arXiv preprint arXiv:2401.15235, 2024.

[7] Roy, A.M., Bhaduri, J., Kumar, T., Raj, K. (2022). WilDect-YOLO: An efficient and robust computer vision-based accurate object localization model for automated endangered wildlife detection. Ecological Informatics, XX, 101919.

[8] Liu, Y., Shao, Z., Hoffmann, N. (2021). Global attention mechanism: Retain information to enhance channel-spatial interactions. Advances in Neural Information Processing Systems.

[9] Redmon, J., Farhadi, A. (2018). YOLOv3: An Incremental Improvement. arXiv.

[10] Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., Chen, L. C. (2018). MobileNetV2: Inverted residuals and linear bottlenecks. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 4510-4520).

[11] Howard, A., Sandler, M., Chu, G., Chen, L. C., Chen, B., Tan, M., ... Adam, H. (2019). Searching for MobileNetV3. In Proceedings of the IEEE/CVF international conference on computer vision (pp. 1314-1324).

[12] Han, S., Pool, J., Tran, J., Dally, W. J. (2015). Learning both weights and connections for efficient neural network. Advances in neural information processing systems, 28.

[13] He, K., Zhang, X., Ren, S., Sun, J. (2016). Deep residual learning for image recognition. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 770-778).

[14] Iandola, F. N., Han, S., Moskewicz, M. W., Ashraf, K., Dally, W. J., Keutzer, K. (2016). SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and ¡0.5 MB model size. arXiv preprint arXiv:1602.07360.

[15] Hinton, G., Vinyals, O., Dean, J. (2015). Distilling the knowledge in a neural network. arXiv preprint arXiv:1503.02531.

[16] Wu, H., Wang, J., Wu, Z., Liu, Y. (2021). Structured pruning for lightweight deep neural networks. IEEE Transactions on Neural Networks and Learning Systems.

[17] Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., ... Adam, H. (2017). MobileNets: Efficient convolutional neural networks for mobile vision applications. arXiv preprint arXiv:1704.04861.

[18] Zhou, A., Yang, Q., Wang, Y. (2020). Model compression with pruning and quantization for edge devices. In Proceedings of IEEE International Conference on Edge Computing.

[19] Lin, J., Rao, Y., Lu, J., Zhou, J. (2019). Runtime neural pruning. Advances in Neural Information Processing Systems, 32.

[20] Li, H., Kadav, A., Durdanovic, I., Samet, H., Graf, H. P. (2016). Pruning filters for efficient convnets. arXiv preprint arXiv:1608.08710.

[21] Rastegari, M., Ordonez, V., Redmon, J., Farhadi, A. (2016). XNOR-Net: ImageNet classification using binary convolutional neural networks. In European conference on computer vision (pp. 525-542). Springer, Cham.

[22] Alvarez, J. M., Salzmann, M. (2016). Learning the number of neurons in deep networks. Advances in neural information processing systems, 29.

[23] Zhang, X., Zhou, X., Lin, M., Sun, J. (2018). ShuffleNet: An extremely efficient convolutional neural network for mobile devices. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 6848-6856).

[24] Liu, Z., Li, J., Shen, Z., Huang, G., Yan, S., Zhang, C. (2019). Learning efficient convolutional networks through network slimming. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 2736-2744).

[25] Ma, N., Zhang, X., Zheng, H. T., Sun, J. (2018). ShuffleNet V2: Practical guidelines for efficient CNN architecture design. In Proceedings of the European conference on computer vision (ECCV) (pp. 116-131).

[26] Tai, C., Xiao, T., Zhang, Y., Wang, X., E, W. (2015). Convolutional neural networks with low-rank regularization. arXiv preprint arXiv:1511.06067.

[27] Luo, J., Wu, J., Lin, W. (2017). ThiNet: A filter level pruning method for deep neural network compression. In Proceedings of the IEEE international conference on computer vision (pp. 5058-5066).

[28] Yang, T., Zhang, X., Yu, J., Sun, J. (2018). CondenseNet: An efficient DenseNet using learned group convolutions. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 2752-2761).

[29] Frankle, J., Carbin, M. (2018). The lottery ticket hypothesis: Finding sparse, trainable neural networks. arXiv preprint arXiv:1803.03635.

[30] Molchanov, P., Tyree, S., Karras, T., Aila, T., Kautz, J. (2016). Pruning convolutional neural networks for resource efficient inference. arXiv preprint arXiv:1611.06440.